# Sprint 15 · Reto «El mejor modelo, demostrado»

**Ficha completa del reto, para consultar.** Para trabajar, el reto en una página. Qué se sube y cómo se evalúa: entrega y rúbrica.

- Volver al README del sprint.
- Si es vuestro primer reto: Cómo se trabaja un reto.

## RETO Ficha del reto

### 1 Qué os pide el cliente

**Qué os pide el cliente.** El cliente (el docente) quiere saber **qué modelo recomendáis para vuestro proyecto y por qué**. No le basta con «el que tiene el número más alto»: quiere saber si la mejora frente al *baseline* (el modelo más simple, que sirve de referencia) **compensa el coste y la complejidad**, si el modelo **funciona con datos que no ha visto** y si vuestros resultados **se pueden repetir** otro día y en otro ordenador. Este sprint es el corazón técnico del proyecto, pero lo que se valora **no es probar muchos modelos**: es **probar con orden, anotar cada prueba y elegir con honestidad**.

**Cómo se trabaja.** Partís de lo que ya tenéis del Sprint 14 (datos, análisis y *baseline*) y de las herramientas del cuaderno de práctica: una función que fija el azar, otra que anota cada prueba en un fichero y la validación cruzada (evaluar varias veces con trozos distintos de los datos). Vosotros decidís **antes** qué modelos probáis y por qué (como mínimo, tres modelos clásicos y uno neuronal, o un asistente LLM/RAG si vuestro brief es el 3), los comparáis **con la misma partición**, **elegís el candidato sin mirar el test** y solo después lo evaluáis en el test **una única vez**. Son tres jornadas: **470 minutos de reto** (130 el día 1, 200 el día 2 y 140 el día 3) más 60 de review y retro. El equipo decide hasta dónde llega: mínimo, estándar o extra.

## 2 · Recursos del reto

| Recurso | Para qué lo usaréis | Dónde está |
|---|---|---|
| **Python 3.11 y las bibliotecas del proyecto** (`pandas`, `scikit-learn`, `matplotlib`; la de redes que usasteis en el Sprint 10) | Entrenar, comparar y guardar los modelos | Colab, o el entorno del equipo con las versiones de vuestro `requirements.txt` |
| **Cuaderno del sprint** `S15_01_seguimiento_experimentos` | Copiar las funciones `fijar_semillas`, `registrar` y `experimento`, y ver cómo se mide con validación cruzada, cómo se detecta una fuga y cómo se evalúa el test una vez | Colab (50 minutos el día 1). MLflow es **opcional** |
| **Plantilla del registro** `registro_experimentos_plantilla.csv` | Solo trae la cabecera de las columnas; es la base de vuestro `registro.csv` | En GitHub: *Raw* → *Download* o copiar el texto |
| **Píldora y apuntes** presentación S15.1 · apuntes | El porqué de cada regla (semillas, fuga, media ± desviación, test una vez), con ejemplos numéricos | En el repositorio del curso |
| **Vuestro trabajo del Sprint 14** | Los datos, el *baseline* y el cuaderno de partida | El repositorio del equipo (`ifcd107-equipo-…`) |

| Recurso | Para qué lo usaréis | Dónde está |
|---|---|---|
| **Cómo entregar en GitHub** | Subir los ficheros a la carpeta `sprint-15/` | Guía de entrega |
| **Autoevaluación y coevaluación** | Cada persona rellena su hoja | Hoja del sprint |
| **Un asistente de IA** (opcional) | Entender errores, pedir pistas y revisar si hay fugas | Con las reglas de la «Ayuda 1» del paso a paso |

## 3 · Guía paso a paso

Son cinco pasos, de **470 minutos en total** (130 el día 1, 200 el día 2 y 140 el día 3, junto a la píldora, la práctica, la tutoría, la entrega y la review). Las jornadas de proyecto duran **6 horas completas (250 minutos de trabajo al día)**: los pasos de esta ficha ocupan unos 200 y los **50 minutos restantes de cada día son «subir de nivel»** (pasar del mínimo al estándar con la rúbrica en la mano). Cada uno dice **qué hacéis** (la lista corta), **cómo se hace** con el detalle, **qué deberíais ver** y **cuándo está hecho**. Las ayudas (IA, errores, atascos) están al final. Si el tiempo aprieta, completad el nivel mínimo y dejad lo demás para casa.

## Paso 1 · Preparar el terreno y repetir el baseline (día 1, 130 minutos)

1. Fijar la semilla, anotar las versiones y crear el registro con la plantilla.
2. Definir la partición común y escribir qué modelos se van a probar y por qué.
3. Repetir el *baseline* y anotarlo como primera fila del registro.
4. Entrenar el primer modelo clásico y anotarlo.

**En sencillo.** Este sprint se parece a un laboratorio: vais a probar varios modelos y hay que **apuntar cada prueba**, porque al tercer día nadie recuerda si aquel 0,75 era con la semilla 1 o la 42. Hoy preparáis el terreno: un cuaderno común, una forma de dejar el azar fijo (la **semilla**), un **cuaderno de notas** en forma de tabla (el **registro**) y un reparto de los datos que no se cambia nunca. Con eso hecho, repetís el modelo más simple (el *baseline*) y entrenáis el primero de verdad. No hace falta entender cómo funciona cada modelo: el código viene de la práctica `S15_01` y solo cambiáis las líneas marcadas.

**Palabras nuevas de este paso**

| Palabra | Qué significa, en sencillo |
|---|---|
| **Semilla** | Un número fijo (42) que hace que el azar dé siempre lo mismo. Con ella, otra persona obtiene exactamente vuestras cifras. |
| **Registro de experimentos** | Una tabla ( `registro.csv` ) con una fila por prueba: quién, qué modelo, con qué ajustes y qué resultado. |
| **Hiperparámetro** | Un ajuste que elegís vosotros antes de entrenar (por ejemplo, la profundidad de un árbol). |
| **Validación cruzada** | Partir el entrenamiento en 5 trozos y entrenar 5 veces, probando cada vez en el trozo que no se ha visto. Da una **media** y una **desviación**. |
| **Test** | Datos apartados que no se miran hasta el final. Se usan **una sola vez**. |
| **Pipeline** | Una cadena de pasos (escalar, codificar, modelo) que se aprende solo con el entrenamiento. |
| **Huella (hash)** | Un código corto que cambia si cambian los datos. |
| **AUC** | Una métrica de 0 a 1 para clasificar: 0,5 es azar y 1 es perfecto. |

**Cómo se hace**

1. **Crear el cuaderno común (5 minutos).** Una persona (la responsable técnica) abre Colab, *Archivo → Cuaderno nuevo*, lo llama `03_modelos.ipynb` y pulsa **Compartir** (arriba a la derecha) para dar acceso de **editor** al resto. Subid también vuestro archivo de datos (panel de la izquierda). Todo el equipo trabaja en **este mismo cuaderno**: lo común arriba, y debajo una sección por persona con su nombre. **No editéis las celdas de otra persona.**
2. **Semilla y versiones (10 minutos).** Si ya hicisteis `S15_01`, copiad de allí estas celdas. En la primera celda de `03_modelos.ipynb` van las importaciones:

```
import os, sys, time, json, random, hashlib
from datetime import datetime
import numpy as np
import pandas as pd
import sklearn
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.dummy import DummyClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.metrics import roc_auc_score
```

   En la segunda, la semilla y las versiones:

```
def fijar_semillas(semilla):
    random.seed(semilla)
    np.random.seed(semilla)
    os.environ["PYTHONHASHSEED"] = str(semilla)
    return np.random.default_rng(semilla)

SEMILLA = 42
rng = fijar_semillas(SEMILLA)

ENTORNO = {"python": sys.version.split()[0], "numpy": np.__version__, "pandas": pd.__version__, "scikit-learn":
sklearn.__version__}
print(json.dumps(ENTORNO, indent=2))
```

   Qué hace cada línea:

   - `import` carga las herramientas que se usarán; **no se cambian**.
   - `fijar_semillas` fija el azar de `random` y de `numpy`; `SEMILLA = 42` es el número. **No lo cambiéis hasta que salga un resultado bonito: nunca.** Es trampa.
   - `ENTORNO` junta las versiones y `print(json.dumps(...))` las escribe. Salen cuatro líneas: `python`, `numpy`, `pandas` y `scikit-learn` con su versión (con las vuestras, que dependen de Colab). Copiadlas en una celda de texto y más tarde en el README.

3. **Crear el registro (10 minutos).** El registro tiene doce columnas, las mismas que la plantilla `registro_experimentos_plantilla.csv` (que solo trae esa cabecera). No hace falta subir la plantilla: este código crea `registro.csv` con esas columnas.

| Columna | Qué se anota |
|---|---|
| `id` | Un número que sube con cada prueba (1, 2, 3…) |
| `fecha`, `responsable` | Cuándo y **quién** hizo la prueba (demuestra que participa todo el equipo) |
| `modelo`, `hiperparametros` | Qué modelo y con qué ajustes |
| `datos_hash`, `semilla` | Una huella corta de los datos y la semilla usada |
| `metrica`, `valor_cv`, `desv_cv` | La métrica que fijasteis en el sprint 14, su media y su desviación en validación cruzada |
| `tiempo_s`, `notas` | Cuánto tardó y lo que habéis aprendido; aquí se marca también si el código lo generó una IA |

```
RUTA_REGISTRO = "registro.csv"
COLUMNAS = ["id", "fecha", "responsable", "modelo", "hiperparametros", "datos_hash", "semilla",
            "metrica", "valor_cv", "desv_cv", "tiempo_s", "notas"]
if os.path.exists(RUTA_REGISTRO):
    os.remove(RUTA_REGISTRO)

def registrar(responsable, modelo, hiperparametros, datos_hash, semilla, metrica, valor_cv, desv_cv, tiempo_s, notas=""):
    existe = os.path.exists(RUTA_REGISTRO)
    n = len(pd.read_csv(RUTA_REGISTRO)) if existe else 0
    fila = pd.DataFrame([{
        "id": n + 1, "fecha": datetime.now().strftime("%Y-%m-%d %H:%M"), "responsable": responsable, "modelo": modelo,
        "hiperparametros": json.dumps(hiperparametros, sort_keys=True), "datos_hash": datos_hash, "semilla": semilla,
        "metrica": metrica, "valor_cv": round(float(valor_cv), 4), "desv_cv": round(float(desv_cv), 4),
        "tiempo_s": round(float(tiempo_s), 2), "notas": notas}], columns=COLUMNAS)
    fila.to_csv(RUTA_REGISTRO, mode="a", header=not existe, index=False)
    return n + 1
```

- `registrar(...)` añade **una fila** al final de `registro.csv`: si el archivo no existe lo crea con la cabecera, y el `id` es el número de filas + 1. **No cambiéis los nombres de las columnas.**
- Las líneas con `os.remove` borran el registro al empezar para que cada ejecución completa del cuaderno lo **rehaga desde cero** (sin filas repetidas). El `registro.csv` final es, por tanto, el resultado de ejecutar el cuaderno entero.

4. **Datos y partición común (20 minutos).** Una sola partición para todo el equipo, que no se cambia. Es la primera del sprint 14: con la misma semilla sale **el mismo test**. El test se aparta ahora y **no se toca hasta el paso 5**.

```
RUTA = "mis_datos.csv"
OBJETIVO = "cancelada"
QUITAR = ["id_reserva", "fecha_reserva", "fecha_llegada"]

df = pd.read_csv(RUTA, parse_dates=["fecha_reserva", "fecha_llegada"])
df["mes_llegada"] = df["fecha_llegada"].dt.month
X = df.drop(columns=QUITAR + [OBJETIVO])
y = df[OBJETIVO]
X_ent, X_test, y_ent, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=SEMILLA)

DATOS_HASH = hashlib.md5(pd.util.hash_pandas_object(df, index=True).values.tobytes()).hexdigest()[:8]
print("Huella de los datos:", DATOS_HASH, "· entrenamiento:", X_ent.shape, "· test (sin tocar):", X_test.shape)
```

- **Cambiad** `RUTA`, `OBJETIVO` (la columna a predecir) y `QUITAR` (identificadores y fechas en bruto: no son pistas válidas). Las dos líneas de `parse_dates` y `mes_llegada` son del ejemplo con fechas: adaptadlas o borradlas.
- `X` son las pistas y `y` la respuesta. `X_ent` es el entrenamiento (con validación cruzada dentro) y `X_test` se guarda.
- `DATOS_HASH` es la huella. Con `reservas_turisdata.csv` sale `Huella de los datos: 800f40b2 · entrenamiento: (4800, 15) · test (sin tocar): (1200, 15)`. Con vuestros datos saldrán otros números.

5. **Escribir el plan (en una celda de texto).** Debajo de lo anterior, una celda de texto con **qué vais a probar y por qué**. Ejemplo: «Regresión logística (simple y explicable), bosque aleatorio y *gradient boosting* (capturan relaciones que no son líneas rectas) y una red neuronal pequeña. Ajustaremos solo en validación: la profundidad del bosque». **No vale «probar 200 combinaciones y quedarse con la mejor».**
6. **El baseline como fila 1 (30 minutos).** Las funciones de comparación (la cadena de pasos, el reparto de la validación cruzada y `experimento`) y el *baseline* del sprint 14:

```
categoricas = X.select_dtypes("object").columns.tolist()
METRICA = "AUC"
SCORING = "roc_auc"

def crear_pipeline(estimador, columnas):
    numericas = [c for c in columnas if c not in categoricas]
    prep = ColumnTransformer([("num", StandardScaler(), numericas), ("cat", OneHotEncoder(handle_unknown="ignore"),
categoricas)])
    return Pipeline([("prep", prep), ("m", estimador)])

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEMILLA)

def experimento(nombre, estimador, hiperparametros, responsable="equipo", notas="", Xd=None):
    Xd = X_ent if Xd is None else Xd
    t0 = time.time()
    puntuaciones = cross_val_score(crear_pipeline(estimador, Xd.columns), Xd, y_ent, cv=cv, scoring=SCORING)
    registrar(responsable, nombre, hiperparametros, DATOS_HASH, SEMILLA, METRICA, puntuaciones.mean(), puntuaciones.std(),
time.time() - t0, notas)
    return puntuaciones.mean(), puntuaciones.std()

media, desv = experimento("Baseline", DummyClassifier(strategy="prior"), {"strategy": "prior"}, responsable="Ana",
notas="referencia mínima")
print(f"Baseline: {media:.3f} ± {desv:.3f}")
```

   - `METRICA` es el nombre que se escribe en el registro y `SCORING` el mismo concepto con el nombre de scikit-learn. **Poned la métrica que fijasteis en el sprint 14**: F1 es `"f1"` y exactitud es `"accuracy"`.
   - `crear_pipeline` convierte los números a una misma escala y los textos en columnas de 0 y 1, **dentro** de la cadena, para que solo se aprenda con el entrenamiento. `cv` define los 5 trozos.
   - `experimento` entrena y evalúa con validación cruzada, mide el tiempo y **registra la fila**. **Cambiad `"Ana"` por vuestro nombre** en el parámetro `responsable`.
   - El `Baseline` con `DummyClassifier(strategy="prior")` no mira las pistas y sale con `Baseline: 0.500 ± 0.000`: eso es el azar. Debe ser la **fila 1**.
7. **El primer modelo (55 minutos).** Una persona entrena el primero mientras las demás preparan el siguiente en su sección. En una celda nueva, con vuestro nombre:

```
media, desv = experimento("RegresionLogistica", LogisticRegression(max_iter=2000), {"C": 1.0}, responsable="Ana")
print(f"Regresión logística: {media:.3f} ± {desv:.3f}")
```

   Con los datos de práctica sale `Regresión logística: 0.755 ± 0.012`. Las cifras de los vuestros serán otras: lo que importa es que **supere al baseline**. Comprobad el registro con `pd.read_csv(RUTA_REGISTRO)`: debe tener las filas 1 (*baseline*) y 2.
8. **Comprobar la semilla.** Elegid *Entorno de ejecución → Reiniciar y ejecutar todo* y mirad que las cifras salen **iguales** que antes. Si cambian, algo usa azar sin semilla.

9. **Si vuestros datos son una serie temporal** (brief 1 o 4), sustituid los apartados 4 y 6 por esto. El reparto es por fechas (los últimos 91 días van al test) y se valida con `TimeSeriesSplit`, que respeta el orden del tiempo:

```
 from sklearn.model_selection import TimeSeriesSplit
from sklearn.dummy import DummyRegressor
from sklearn.linear_model import Ridge
from sklearn.ensemble import HistGradientBoostingRegressor
from sklearn.metrics import mean_absolute_error

df = pd.read_csv("mis_datos.csv", parse_dates=["fecha"]).sort_values("fecha")
df["mes"] = df["fecha"].dt.month
df["dia_semana"] = df["fecha"].dt.dayofweek
X = df[["mes", "dia_semana", "festivo"]]
y = df["ocupacion_pct"]
DIAS = 91
X_ent, X_test = X.iloc[:-DIAS], X.iloc[-DIAS:]
y_ent, y_test = y.iloc[:-DIAS], y.iloc[-DIAS:]
print(X_ent.shape, X_test.shape)
DATOS_HASH = hashlib.md5(pd.util.hash_pandas_object(df, index=True).values.tobytes()).hexdigest()[:8]

def crear_pipeline(estimador, columnas):
    return Pipeline([("prep", StandardScaler()), ("m", estimador)])

cv = TimeSeriesSplit(n_splits=5)

def experimento(nombre, estimador, hiperparametros, responsable="equipo", notas="", Xd=None):
    Xd = X_ent if Xd is None else Xd
    t0 = time.time()
    errores = -cross_val_score(crear_pipeline(estimador, Xd.columns), Xd, y_ent, cv=cv, scoring="neg_mean_absolute_error")
    registrar(responsable, nombre, hiperparametros, DATOS_HASH, SEMILLA, "MAE", errores.mean(), errores.std(), time.time()
- t0, notas)
    return errores.mean(), errores.std()

errores = []
for ent, val in cv.split(X_ent):
    errores.append(mean_absolute_error(y_ent.iloc[val], y_ent.shift(7).iloc[val]))
registrar("Ana", "Baseline_semana_pasada", {"desfase_dias": 7}, DATOS_HASH, SEMILLA, "MAE", np.mean(errores),
np.std(errores), 0, "igual que la semana pasada")
media, desv = experimento("Ridge", Ridge(), {"alpha": 1.0}, responsable="Ana")
print(f"MAE de la semana pasada: {np.mean(errores):.2f} ± {np.std(errores):.2f} · Ridge: {media:.2f} ± {desv:.2f}")
```

   Cambiad el nombre del archivo, `ocupacion_pct` (lo que predecís) y las pistas de `X`. En series la métrica es un **error** (MAE), y cuanto **menor**, mejor; el signo menos pasa a positivo lo que `scikit-learn` devuelve como negativo. Con `ocupacion_diaria.csv` salen `(1005, 3) (91, 3)` y `MAE de la semana pasada: 4.77 ± 0.36 · Ridge: 8.26 ± 3.88` (dos filas nuevas en el registro, la 1 y la 2).

**Qué deberíais ver.** Un `registro.csv` con al menos 3 filas (*baseline*, primer modelo y alguna prueba más) y las doce columnas rellenas; el cuaderno ejecutándose de arriba abajo con **las mismas cifras en dos ejecuciones seguidas** (señal de que la semilla funciona); y el *baseline* con la métrica más baja o igual al azar (AUC 0,5 en clasificación).

**Hecho cuando:** hay semilla, versiones, partición común, plan escrito y el registro tiene el *baseline* y al menos un modelo más.

## Si algo falla

| Veis... | Qué significa | Qué hacer |
|---|---|---|
| `NameError: name 'experimento' is not defined` | La celda donde se define no se ha ejecutado, o Colab se reinició. | Ejecutad desde la primera celda hacia abajo (*Entorno de ejecución → Ejecutar todo*). |

| Veis… | Qué significa | Qué hacer |
|---|---|---|
| `FileNotFoundError: [Errno 2] No such file or directory: 'mis_datos.csv'` | El archivo no está subido en esta sesión de Colab, o `RUTA` no coincide. | Subidlo otra vez con el panel y copiad el nombre exacto. Los archivos subidos se pierden cuando Colab se reinicia. |
| `KeyError: "['cancelado'] not found in axis"` (con otros nombres) | Un nombre de `OBJETIVO` o de `QUITAR` no es exacto. | Escribid `df.columns` y copiad los nombres tal cual. |
| `ValueError: All the 5 fits failed.` y, más abajo, `Input X contains NaN` | Hay casillas vacías en columnas numéricas y el modelo no las admite. | Rellenadlas o quitad esas filas dentro de la cadena de pasos (un imputador en el `Pipeline`) o probad `HistGradientBoostingClassifier`, que las acepta. |
| `ValueError: The least populated classes in y have only 1 member, which is too few` | Una clase tiene una sola fila y `stratify` no puede repartirla. | Quitad `stratify=y` o agrupad esa clase con otra, y apuntadlo en las notas. |
| El registro tiene filas repetidas o empieza en un `id` raro | Se ejecutó una celda de `experimento` varias veces sin reiniciar. | *Reiniciar y ejecutar todo*: el registro se rehace desde cero. |
| Cada vez que ejecutáis salen cifras distintas | Falta fijar la semilla en algún paso. | Comprobad que `SEMILLA` se usa en `train_test_split`, `cv` y los modelos (`random_state=SEMILLA`). |

## Paso 2 · Probar los modelos clásicos y el neuronal (día 2, 140 minutos)

1. Entrenar al menos 3 modelos clásicos con la misma partición y anotar cada prueba.
2. Entrenar al menos un modelo neuronal (o el asistente LLM/RAG en el brief 3).
3. Ajustar 1–2 hiperparámetros clave de cada familia, solo con validación.
4. Comprobar que cada persona ha anotado al menos una fila.

**En sencillo.** Hoy se prueba, se apunta y se compara. Cada persona entrena un modelo o ajusta uno, y **cada prueba se anota nada más terminarla** con la función `experimento` que dejasteis hecha en el paso 1: hace todo el trabajo (entrenar, medir y escribir la fila del registro). Es como una carrera con cronometraje: todos corren en la misma pista (la misma partición) y se apunta cada tiempo, también los malos. No hace falta saber cómo funciona cada modelo por dentro: cambiáis el nombre del modelo, el responsable y, si toca, un ajuste.

**Palabras nuevas de este paso**

| Palabra | Qué significa, en sencillo |
|---|---|
| **Modelo clásico** | Un modelo de aprendizaje automático sin redes neuronales: regresión, bosque aleatorio, *gradient boosting*. |
| **Bosque aleatorio** | Muchos árboles de decisión que votan. Ajuste típico: `max_depth` (profundidad de cada árbol). |
| **Gradient boosting** | Árboles que se entrenan uno tras otro, cada uno corrigiendo los errores del anterior. |
| **Red neuronal** | Un modelo hecho de capas de «neuronas» que aprende combinaciones de las pistas. |
| **Sobreajuste** | Que el modelo se aprenda de memoria el entrenamiento y falle con datos nuevos. |
| **Subconjunto** | Una parte de los datos para que una prueba pesada tarde menos. |

**Cómo se hace**

1. **Repartir (5 minutos).** Un reparto posible si sois 3 o 4:

| Persona | Qué prueba | Minutos |
|---|---|---|
| A | Segundo y tercer modelo clásico (por ejemplo, bosque y boosting) | 50 |
| B | Modelo neuronal (o RAG/LLM en el brief 3) | 60 |
| C | Ajuste de 1–2 hiperparámetros de las familias de A (profundidad, número de árboles…) | 40 |
| D (si hay 4) | Repetir una prueba de otra persona con su código para comprobar que sale igual | 40 |

   Cada persona escribe en **su sección** del cuaderno, con una celda de texto con su nombre como título, y **cambia `responsable="..."` por su nombre** en cada llamada.

2. **Persona A: bosque y boosting.** En una celda nueva:

```
media, desv = experimento("RandomForest", RandomForestClassifier(n_estimators=200, min_samples_leaf=5,
random_state=SEMILLA, n_jobs=2),
                          {"n_estimators": 200, "min_samples_leaf": 5}, responsable="Luis")
print(f"Bosque aleatorio: {media:.3f} ± {desv:.3f}")

media, desv = experimento("HistGradientBoosting", HistGradientBoostingClassifier(random_state=SEMILLA),
                          {"defecto": True}, responsable="Luis")
print(f"Gradient boosting: {media:.3f} ± {desv:.3f}")
```

   - El primer argumento es el **nombre** que verá el registro: el mismo modelo debe llamarse **siempre igual**.
   - El segundo es el modelo con sus ajustes; el tercero, un diccionario `{}` con esos mismos ajustes, para que quede escrito (`{"defecto": True}` significa «sin tocar nada»).
   - Cada llamada entrena 5 veces (validación cruzada), escribe una fila en el registro y devuelve la media y la desviación. Con los datos de práctica salen `Bosque aleatorio: 0.739 ± 0.013` y `Gradient boosting: 0.727 ± 0.011`; el boosting puede tardar de unos segundos a casi medio minuto.

3. **Persona C: ajustar sin trampa.** Probad 2 o 3 valores de un ajuste y **anotad cada valor como una fila**:

```
for d in (3, 6, 10):
    media, desv = experimento(f"RandomForest_prof{d}",
                              RandomForestClassifier(n_estimators=200, min_samples_leaf=5, max_depth=d,
random_state=SEMILLA, n_jobs=2),
                              {"n_estimators": 200, "min_samples_leaf": 5, "max_depth": d}, responsable="Marta",
notas="ajuste de profundidad")
    print(f"profundidad {d}: {media:.3f} ± {desv:.3f}")
```

   `for d in (3, 6, 10)` repite el bloque con `d` igual a 3, luego 6 y luego 10; `f"...{d}"` pone el valor en el nombre. Salen aproximadamente `profundidad 3: 0.742 ± 0.013`, `profundidad 6: 0.745 ± 0.013` y `profundidad 10: 0.742 ± 0.013`: **casi iguales**, y es normal. Anotad también lo que no mejora («empeoró al aumentar la profundidad: sobreajuste»): también es información. **Solo con validación; el test ni se toca.**

4. **Persona B: el modelo neuronal.** La opción más sencilla es la red de scikit-learn, que cabe en la misma función y la misma validación cruzada:

```
from sklearn.neural_network import MLPClassifier

media, desv = experimento("RedNeuronal_MLP", MLPClassifier(hidden_layer_sizes=(32, 16), early_stopping=True, max_iter=300,
random_state=SEMILLA),
                          {"hidden_layer_sizes": [32, 16], "early_stopping": True, "max_iter": 300}, responsable="Pablo",
notas="red neuronal de scikit-learn")
print(f"Red neuronal: {media:.3f} ± {desv:.3f}")
```

   `hidden_layer_sizes=(32, 16)` son dos capas de 32 y 16 neuronas y `early_stopping=True` para de entrenar cuando deja de mejorar. Con los datos de práctica sale aproximadamente `Red neuronal: 0.738 ± 0.019`. Si preferís la red de Keras del sprint 10, copiad `construir_mlp` y el entrenamiento de `S10_04` y anotad la fila a mano con `registrar(...)`, poniendo la métrica de **una sola partición de validación** en `valor_cv`, un `0` en `desv_cv` y en `notas` «una sola partición de validación». En el brief 2 (imágenes) y el brief 3 (asistente), mirad `S11_02` y `S12_06` y anotad también con `registrar`. Si tarda mucho o usa una API de pago, **trabajad con un subconjunto de datos y anotad el coste en `notas`**.

5. **Persona D (si hay cuatro): repetir una prueba.** Copiad en vuestra sección la celda de otra persona (sin cambiar nada salvo `responsable` y `notas="repetición de la prueba de Luis"`) y comprobad que da **las mismas cifras**. Si no, hay azar sin semilla.

6. **Si una prueba la ha generado una IA,** escribidlo en `notas`, por ejemplo `notas="código generado con IA, revisado a mano"`. **Que cada persona toque algo:** si alguien se queda sin fila, que pruebe un ajuste de otra persona y lo anote con su nombre.

7. **Comprobar el registro (10 minutos).** El registro de cada persona vive en su propia sesión de Colab. Para ver todas las filas juntas, la persona con el rol técnico elige *Entorno de ejecución → Reiniciar y ejecutar todo*: se ejecutan todas las secciones, una tras otra, y el registro se rehace completo. Después ejecuta:

```
registro = pd.read_csv(RUTA_REGISTRO)
print(len(registro), "filas")
print(registro["responsable"].value_counts())
```

   `value_counts()` cuenta cuántas filas tiene cada nombre. Con las pruebas de ejemplo de arriba (ocho en total) salen `8 filas` y `Marta 3`, `Ana 2`, `Luis 2`, `Pablo 1`. Cada persona del equipo debe aparecer **al menos una vez**.

**Qué deberíais ver.** Un `registro.csv` con unas 8–12 filas de personas distintas, con `valor_cv` y `desv_cv` rellenos. Si dos modelos tienen medias casi iguales y desviaciones parecidas, es normal.

**Hecho cuando:** hay *baseline* y 3 modelos clásicos y 1 neuronal en el registro con la misma partición, y cada persona tiene al menos una fila.

## Si algo falla

| Veis... | Qué significa | Qué hacer |
|---|---|---|
| `NameError: name 'RandomForestClassifier' is not defined` (o `MLPClassifier`) | Falta importar el modelo, o la celda de importaciones no se ejecutó. | Ejecutad la primera celda del cuaderno; para la red, la línea `from sklearn.neural_network import MLPClassifier` va antes de usarla. |
| `NameError: name 'experimento' is not defined` | Vuestra sesión de Colab no ha ejecutado las celdas comunes. | *Entorno de ejecución → Ejecutar todo* hasta la celda de `experimento` y volved a vuestra sección. |
| Solo veis vuestras filas en el registro | Cada persona tiene su propia sesión de Colab y su propio `registro.csv`. | Que la persona técnica haga *Reiniciar y ejecutar todo* en el cuaderno común y comparta el resultado. |

| Veis... | Qué significa | Qué hacer |
|---|---|---|
| Todas las pruebas dan casi lo mismo | Hay poca diferencia real entre los modelos. | Es normal: dejadlo escrito y decidid en el paso 4 por simplicidad y coste. |
| Una celda tarda mucho o Colab se queda sin memoria | El modelo o los datos son pesados. | Probad con un subconjunto (por ejemplo, la mitad de las filas), reducid `n_estimators` y anotad el coste en `notas`. |
| Una persona sin ninguna fila | No ha ejecutado `experimento` con su nombre. | Que pruebe un ajuste distinto de otra persona y lo anote con su `responsable`. |

## Paso 3 · Comprobar que no hay fugas y tutoría (día 2, 60 minutos)

1. Contestar la lista de fugas en una celda de texto de `03_modelos.ipynb`.
2. Corregir lo que falle y repetir las pruebas afectadas.
3. Ir a la tutoría técnica de 20 minutos con el docente.

**En sencillo.** Una **fuga de datos** es una trampa involuntaria: el modelo ve una pista que, en la vida real, todavía no existiría. Es como un examen en el que la respuesta está escrita en el enunciado: sacas un 10, pero no has aprendido nada. Por eso, si un resultado parece **demasiado bueno para ser verdad**, hay que sospechar. Hoy contestáis cinco preguntas por escrito, limpiáis el registro y habláis con el docente. No hay que programar mucho: una demostración opcional y tres líneas para revisar el registro.

**Palabras nuevas de este paso**

| Palabra | Qué significa, en sencillo |
|---|---|
| **Fuga de datos** | Usar, sin querer, información del futuro o de la propia respuesta para predecir. |
| **«En el momento de decidir»** | El instante en que se usaría el modelo. Solo vale lo que ya se conoce entonces. |
| **Variable de fuga** | Una columna que solo existe después del suceso (por ejemplo, `reembolso_emitido`, que solo hay si la reserva se canceló). |
| **Duplicados** | Filas que son la misma prueba repetida. |
| **Tutoría técnica** | Una conversación de 20 minutos con el docente para revisar vuestro trabajo. |

**Cómo se hace**

1. **La lista de fugas (20 minutos).** En `03_modelos.ipynb`, añadid una celda de texto (botón **+ Texto**) titulada «Comprobación de fugas» y copiad estas cinco preguntas. Contestad **Sí / No con una frase** a cada una:
    1. ¿Las variables se conocerían **en el momento de decidir**?
    2. ¿El escalado, la imputación y la codificación se aprenden **solo con el entrenamiento** (dentro de un `Pipeline`)?
    3. ¿La partición respeta el **tiempo** o los **grupos**?
    4. ¿Se ha ajustado algo mirando el test? Si sí, hay que rehacer la evaluación.
    5. ¿La métrica es **demasiado buena para ser verdad**? Sospechad.

    Ejemplo de respuestas con los datos de práctica (las vuestras dependen de vuestros datos): «1 · Sí: antelación, noches, canal, depósito… se conocen al reservar; quitamos el identificador y las fechas en bruto. 2 · Sí: usamos `crear_pipeline`, que aprende dentro de cada trozo de la validación cruzada. 3 · No hay series ni fotos del mismo objeto: reparto aleatorio estratificado. 4 · No: `X_test` no se ha usado. 5 · No: un AUC de 0,75 es creíble en este problema».

2. **Ver cómo suena una fuga (5 minutos, recomendable).** Para reconocer una, provocad una a propósito en vuestra sección, con la columna inventada `reembolso_emitido`, que «sabe» la respuesta:

```
X_fuga = X_ent.copy()
X_fuga["reembolso_emitido"] = ((y_ent == 1) & (rng.random(len(y_ent)) < 0.9)).astype(int)
media, desv = experimento("LogisticaConFuga", LogisticRegression(max_iter=2000), {"C": 1.0}, responsable="Ana",
                          notas="ALARMA: AUC sospechosamente alto (fuga: reembolso_emitido)", Xd=X_fuga)
print(f"AUC con la variable con fuga: {media:.3f}")
```

   Qué hace cada línea:

   - `X_fuga` es una copia de las pistas a la que se añade una columna que vale 1 casi siempre que la reserva se canceló: es la respuesta disfrazada de pista.
   - `Xd=X_fuga` le dice a `experimento` que use esa tabla en lugar de `X_ent`. La fila queda en el registro con el nombre `LogisticaConFuga` y la nota `ALARMA`.

   Con los datos de práctica sale aproximadamente `AUC con la variable con fuga: 0.975`, frente a ≈ 0,75 sin ella: **demasiado bueno**. Este modelo **no se puede elegir** (en el paso 4 se excluye por llevar «Fuga» en el nombre); sirve para contar en la *review* **una fuga que detectasteis**.

3. **Si una respuesta falla, corregid y repetid.** Si descubrís una variable que no se conocería al decidir, quitadla de `X` y repetid las pruebas afectadas. **No borréis la celda antigua**: dejadla y añadid en su `notas=` que tenía una fuga (por ejemplo, `notas="tenía fuga: variable X; repetida sin ella"`). Así el registro cuenta lo que pasó, y sirve para la rúbrica.

4. **Tutoría técnica (20 minutos).** El docente indica la franja de cada equipo. Llevad la tabla del registro, que se muestra con:

```
registro = pd.read_csv(RUTA_REGISTRO)
registro[["id", "responsable", "modelo", "valor_cv", "desv_cv", "tiempo_s", "notas"]]
```

   Contad **qué comparación estáis haciendo** y **qué fuga podría haber**. Anotad las respuestas del docente en una celda de texto («Tutoría»).

5. **Limpiar el registro (15 minutos).** Revisad columnas vacías, filas repetidas y nombres distintos para el mismo modelo:

```
print(registro.isna().sum())
print(registro["modelo"].value_counts())
print("Repetidas:", registro.duplicated(subset=["modelo", "hiperparametros"]).sum())
```

   - `isna().sum()` cuenta las casillas vacías por columna: solo `notas` puede tener algunas; las demás deben estar a 0.
   - `value_counts()` cuenta cuántas filas tiene cada nombre de modelo: si veis `RandomForest` y `randomforest`, son el mismo modelo con dos nombres; corregid el nombre en la celda donde lo escribisteis.
   - La última línea cuenta pruebas con el mismo modelo y los mismos ajustes. Con las pruebas de ejemplo sale `Repetidas: 0`; las repeticiones hechas a propósito (persona D) llevan una nota que lo dice y no cuentan como error.

**Qué deberíais ver.** La celda «Comprobación de fugas» con cinco respuestas, un registro sin columnas vacías (salvo `notas`) ni filas repetidas por error y, si ha habido una fuga, una fila anotada que lo explica. Ninguna prueba con un resultado casi perfecto sin explicación.

**Hecho cuando:** la lista está contestada, el registro está limpio y la tutoría ha quedado anotada.

## Si algo falla

| Veis... | Qué significa | Qué hacer |
|---|---|---|
| Un modelo da un AUC de 0,97 o más a la primera | Casi seguro, hay una fuga. | Mirad la lista de columnas: ¿alguna solo existe después de la respuesta? Quitadla y repetid. |

| Veis... | Qué significa | Qué hacer |
|---|---|---|
| `NameError: name 'rng' is not defined` | No se ejecutó la celda de `fijar_semillas`. | Ejecutad desde la primera celda hacia abajo. |
| El registro tiene dos o tres filas `LogisticaConFuga` | Se ejecutó la celda de la demostración varias veces. | *Reiniciar y ejecutar todo* rehace el registro y deja una sola fila. |
| `value_counts()` muestra `RandomForest` y `randomforest` como dos modelos | El mismo modelo se anotó con dos nombres distintos. | Corregid el nombre en la celda donde lo escribisteis y volved a ejecutar todo. |
| `FileNotFoundError: [Errno 2] No such file or directory: 'registro.csv'` | Se reinició Colab y el archivo no se ha vuelto a crear. | *Reiniciar y ejecutar todo*, que rehace el registro desde cero. |
| No sabéis si una variable es una fuga | Cuesta decidir qué se conoce «al decidir». | Preguntaos: «¿La tendría ya en la mano el día que uso el modelo?». Si no, es fuga: preguntadlo en la tutoría. |

## Paso 4 · Elegir el candidato sin tocar el test (día 3, 70 minutos)

1. Calcular media ± desviación de cada modelo.
2. Comparar teniendo en cuenta la incertidumbre y añadir coste y complejidad.
3. Elegir el candidato y escribir la justificación de una página.

**En sencillo.** Ya tenéis un montón de pruebas apuntadas; hoy toca **elegir cuál os lleváis**, sin mirar el test. No gana siempre el que tiene el número más alto: si dos modelos van casi igual, la diferencia puede ser **ruido** (casualidad del reparto), y entonces conviene el más simple, el más rápido o el más fácil de explicar. Es como elegir coche: dos modelos con casi el mismo consumo, pero uno es mucho más barato de mantener. El código es corto: ordenar la tabla del registro y comparar. Lo difícil y valioso es **escribir por qué**.

**Palabras nuevas de este paso**

| Palabra | Qué significa, en sencillo |
|---|---|
| **Media ± desviación** | Resultado medio de las 5 pruebas de la validación cruzada y cuánto se separan entre sí. `0,755 ± 0,012` significa «0,755, con un margen de ≈ 0,012». |
| **Solapar / ruido** | Dos modelos se solapan si la diferencia entre sus medias es **menor que su desviación**: no se puede afirmar que uno sea mejor. |
| **Candidato** | El modelo que elegís para evaluar en el test y llevar al demostrador. |
| **Explicabilidad** | Lo fácil que es contar por qué el modelo decide lo que decide. |
| **Coste y complejidad** | Tiempo de entrenamiento y lo difícil que es mantenerlo o explicarlo. |
| **Justificación** | Una página que cuenta qué modelo elegís y por qué, para que lo entienda quien no programa. |

**Cómo se hace**

1. **La tabla ordenada (15 minutos, con los puntos 2 y 3).** En `03_modelos.ipynb`, una celda nueva, tras *Reiniciar y ejecutar todo* para tener el registro completo. Quitad las pruebas con fuga:

```
tabla = pd.read_csv(RUTA_REGISTRO)
legitimos = tabla[~tabla["modelo"].str.contains("Fuga")]
ordenada = legitimos.sort_values("valor_cv", ascending=False)[["id", "responsable", "modelo", "valor_cv", "desv_cv",
"tiempo_s"]]
print(ordenada.to_string(index=False))
```

   - `legitimos` conserva las filas cuyo nombre **no** contiene «Fuga» (la `~` significa «no»).
   - `sort_values("valor_cv", ascending=False)` ordena de mayor a menor. **Si vuestra métrica es un error (MAE), el mejor es el menor: usad `ascending=True`.**
   - Muestra una fila por prueba con quién la hizo, el modelo, la media, la desviación y el tiempo.

2. **Ver si la diferencia es ruido.** Esta función aplica la regla del solape, y el bucle compara cada modelo con el mejor:

```
def parecen_iguales(a, b):
    return abs(a["valor_cv"] - b["valor_cv"]) < max(a["desv_cv"], b["desv_cv"])

mejor = legitimos.loc[legitimos["valor_cv"].idxmax()]
for _, fila in legitimos.iterrows():
    if fila["modelo"] != mejor["modelo"]:
        print(fila["modelo"], "→ ¿diferencia dentro del ruido?", parecen_iguales(mejor, fila))
```

   - `parecen_iguales` devuelve `True` si la diferencia de medias es menor que la mayor de las dos desviaciones.
   - `idxmax()` da la fila con la mayor media (con MAE, usad `idxmin()`). Cada línea impresa dice si ese modelo está dentro del ruido del mejor. **Es una regla práctica, no una prueba estadística.**

   Con las pruebas de ejemplo (datos de práctica) el mejor es la regresión logística (0,755 ± 0,012): el *baseline*, el bosque sin ajustar, el boosting y el bosque de profundidad 3 salen `False` (hay diferencia clara) y el bosque de profundidad 6 y 10 y la red neuronal salen `True` (**solapan**: la diferencia puede ser ruido).

3. **(Opcional) Un gráfico de la comparación.** Con barras de error de un vistazo:

```
import matplotlib.pyplot as plt

t = legitimos.sort_values("valor_cv")
fig, ax = plt.subplots(figsize=(7, 3.5))
ax.barh(t["modelo"], t["valor_cv"], xerr=t["desv_cv"], color="tab:blue", capsize=4)
ax.axvline(0.5, color="gray", linestyle="--", label="Azar (AUC 0,5)")
ax.set_xlim(0.4, 0.85)
ax.set_xlabel("AUC medio en validación cruzada (± desviación)")
ax.set_title("Comparativa de modelos (validación cruzada, sin usar el test)")
ax.legend(loc="lower right")
plt.tight_layout()
plt.show()
```

   Sale una barra horizontal por modelo con un «bigote» que marca su desviación y una línea discontinua en 0,5 (el azar). Si los bigotes de dos barras se solapan, esos modelos son indistinguibles. Con un MAE, quitad la línea del azar y el `set_xlim`.

4. **Coste y complejidad (15 minutos).** Añadid a la comparación: el **tiempo de entrenamiento** (`tiempo_s`), lo difícil que es **mantener** el modelo y lo fácil que es **explicar** su decisión (por ejemplo, la regresión logística tiene un peso por variable que se puede leer; una red neuronal es más difícil de explicar). Escribidlo en una tabla de cuatro columnas en la justificación (el punto 6).

5. **Elegir (15 minutos).** Reglas: se elige con la **validación**, **no se mira el test**; si dos modelos van casi igual, **elegid el más simple y explicadlo**. Una frase del tipo: «Elegimos la regresión logística (0,755 ± 0,012) frente al bosque de profundidad 6 (0,745 ± 0,013): la diferencia puede ser ruido y la logística es mucho más rápida y fácil de explicar» (es el ejemplo con datos de práctica).
6. **Escribir `justificacion_candidato.md` (25 minutos).** En `sprint-15` de GitHub: *Add file → Create new file*, nombre `justificacion_candidato.md`. Una página con esta tabla, rellenada con **vuestras cifras**, y unas líneas debajo que digan por qué gana:

   | Modelo | Métrica (media ± desv.) | Tiempo | Explicabilidad | Comentario |
   |---|---|---|---|---|
   | Baseline | 0,500 ± 0,000 | ≈ 0,1 s | — | Azar: no mira las pistas |
   | Regresión logística | 0,755 ± 0,012 | ≈ 0,1 s | Alta: cada variable tiene un peso | **Candidata** |
   | Bosque (profundidad 6) | 0,745 ± 0,013 | ≈ 2 s | Media | Solapa con la logística: puede ser ruido |
   | Red neuronal (MLP) | 0,738 ± 0,019 | ≈ 1 s | Baja | Solapa con la logística; más difícil de explicar |

   Debajo, 3–5 líneas con: qué modelo elegís, **si la diferencia con el siguiente puede ser ruido**, el coste y el beneficio, y qué no sabéis. Dejad ya un hueco para el resultado en test y para la «Revisión con la rúbrica» del paso 5. Si vais a por el nivel extra, añadid una **hipótesis de por qué gana**: por ejemplo, «los datos tienen relaciones no lineales entre antelación y canal que un árbol captura mejor que una regresión» (ejemplo de redacción; la vuestra se contrasta con la explicación del paso 5).

**Qué deberíais ver.** Una tabla ordenada con una fila por prueba legítima, un candidato elegido y una frase del tipo «Elegimos X (0,71 ± 0,02) frente a Y (0,72 ± 0,04): la diferencia puede ser ruido y X es mucho más rápido» (cifras de ejemplo). Y `justificacion_candidato.md` en `sprint-15`.

**Hecho cuando:** `justificacion_candidato.md` tiene la tabla comparativa y el candidato razonado, y nadie ha mirado el test.

### Si algo falla

| Veis... | Qué significa | Qué hacer |
|---|---|---|
| `NameError: name 'RUTA_REGISTRO' is not defined` | Colab se reinició y las celdas comunes no se ejecutaron. | *Reiniciar y ejecutar todo*; así se rehace el registro. |
| `KeyError: 'valor_cv'` | El nombre de la columna está mal escrito, o la tabla no es el registro. | Escribid `tabla.columns` y comprobad que salen las doce columnas. |
| La fila `LogisticaConFuga` sale la primera | Falta el filtro `legitimos` o se ha usado `tabla` en vez de `legitimos`. | Usad `legitimos` en todas las líneas siguientes. |
| El «mejor» tiene la métrica más baja | Con un error (MAE) el mejor es el menor. | Cambiad `ascending=True` y `idxmin()`. |
| Dos modelos empatan | Es normal cuando solapan. | Elegid el más simple, rápido o fácil de explicar, y escribidlo. |
| Alguien ha mirado el test para decidir | El test deja de ser una prueba limpia. | Decidid solo con validación, anotadlo en la justificación y no volváis a elegir con él. |

## Paso 5 · Test una sola vez, explicación, revisión y entrega (día 3, 70 minutos)

1. Evaluar el candidato en el test, una sola vez.

2. Guardar el modelo con `metadatos.json` y hacer la primera explicación.
3. Revisar con la rúbrica y anotar el resultado.
4. Subir los ficheros a `sprint-15/`.

**En sencillo.** Es el día de la verdad. Hasta ahora el test estaba guardado en un cajón; hoy lo abrís **una sola vez** para ver cómo se portaría el modelo elegido con datos que nunca ha visto. Es como el examen final: no se puede repetir para sacar mejor nota. Después guardáis el modelo en un archivo (para que otra persona lo use sin reentrenarlo), hacéis un primer gráfico de **qué pistas pesan más** en sus decisiones, os revisáis entre vosotros y subís todo. Hay código, pero cada bloque es corto y viene probado.

**Palabras nuevas de este paso**

| Palabra | Qué significa, en sencillo |
|---|---|
| **Test, una sola vez** | Se evalúa el candidato en el test después de elegirlo. Si el resultado no gusta, **no se vuelve a elegir con él**. |
| **`joblib`** | Una herramienta para guardar un modelo entrenado en un archivo (`.joblib`) y volver a abrirlo después. |
| **Metadatos (`metadatos.json`)** | Un archivo de texto con los datos del modelo: qué modelo es, cuándo se entrenó, versiones, resultados y variables. |
| **Importancia por permutación** | Se desordena una variable y se mira cuánto empeora el modelo: cuanto más empeora, más pesa esa variable. |
| **Explicabilidad** | Poder contar qué pistas usa el modelo y por qué decide lo que decide. |
| **Commit** | Cada subida a GitHub con su mensaje y su autor. |

**Cómo se hace**

1. **Test, una vez (10 minutos).** En una celda nueva, con el candidato que elegisteis en el paso 4 (aquí la regresión logística):

```
candidato = LogisticRegression(max_iter=2000)
final = crear_pipeline(candidato, X_ent.columns).fit(X_ent, y_ent)
auc_test = roc_auc_score(y_test, final.predict_proba(X_test)[:, 1])
print(f"AUC en test: {auc_test:.3f}")
```

   Qué hace cada línea:

   - `candidato` es el modelo que elegisteis (**cambiadlo** por el vuestro, con los mismos ajustes que anotasteis en el registro). `crear_pipeline(...).fit(X_ent, y_ent)` lo entrena con **todo** el entrenamiento.
   - `roc_auc_score(y_test, final.predict_proba(X_test)[:, 1])` calcula el AUC en el test (con series, `mean_absolute_error(y_test, final.predict(X_test))`).

   Con los datos de práctica sale `AUC en test: 0.745`, muy cerca del 0,755 de la validación cruzada: es lo esperable. Si el test saliera muy por debajo (más de unas seis centésimas), sospechad sobreajuste o fuga. Copiad el resultado a la justificación. **No cambiéis de candidato después de verlo**: si no os gusta, anotadlo y explicadlo.

2. **Guardar el modelo y los metadatos (10 minutos).**

```
import joblib
from datetime import date

joblib.dump(final, "modelo_candidato.joblib")
metadatos = {
    "modelo": "RegresionLogistica",
    "entrenado": str(date.today()),
    "semilla": SEMILLA,
    "datos_hash": DATOS_HASH,
    "metrica": METRICA,
    "valor_validacion": round(float(tabla.loc[tabla["modelo"] == "RegresionLogistica", "valor_cv"].iloc[0]), 3),
    "valor_test": round(float(auc_test), 3),
    "variables": list(X_ent.columns),
    "versiones": ENTORNO,
}
with open("metadatos.json", "w", encoding="utf8") as f:
    json.dump(metadatos, f, ensure_ascii=False, indent=2)
print(open("metadatos.json").read())
```

Qué hace cada línea:

- `joblib.dump(final, ...)` guarda el modelo **entero** (con su escalado y su codificación) en `modelo_candidato.joblib`; con los datos de práctica pesa unos 5 KB.
- El diccionario `metadatos` recoge versión de las bibliotecas (`ENTORNO`), fecha, semilla, huella de los datos, métrica en validación y en test, y los nombres de las variables. **Cambiad `"RegresionLogistica"` (dos veces) por el nombre de vuestro candidato en el registro.**
- `json.dump(...)` lo escribe en `metadatos.json` y el último `print` lo enseña.

Salen las líneas del archivo, con `"valor_validacion": 0.755` y `"valor_test": 0.745` en el ejemplo. En el brief 3, guardad el asistente tal como lo ejecutáis y apuntad en `metadatos.json` el modelo de lenguaje, los parámetros y el modo de recuperar los textos. Si el modelo es una red de Keras, se guarda con el formato de la red (`.keras`).

3. **Primera explicación (20 minutos).** Importancia de variables por permutación, calculada sobre el **entrenamiento** (para no usar el test una segunda vez):

```
from sklearn.inspection import permutation_importance
import matplotlib.pyplot as plt

resultado = permutation_importance(final, X_ent, y_ent, scoring="roc_auc", n_repeats=5, random_state=SEMILLA)
importancias = pd.Series(resultado.importances_mean, index=X_ent.columns).sort_values(ascending=False)
print(importancias.head(8).round(3))

fig, ax = plt.subplots(figsize=(7, 4))
importancias.head(8)[::-1].plot.barh(ax=ax, color="tab:blue")
ax.set_xlabel("Caída del AUC al desordenar la variable")
ax.set_title("Variables que más pesan en el modelo candidato")
plt.tight_layout()
plt.savefig("explicacion_modelo.png", dpi=150)
plt.show()
```

   Qué hace cada línea:

   - `permutation_importance` desordena cada variable 5 veces (`n_repeats=5`) y mide cuánto cae el AUC. Con series, `scoring="neg_mean_absolute_error"`.
   - `pd.Series(...).sort_values(...)` ordena las variables de más a menos importante; `head(8)` se queda con las ocho primeras.
   - `plt.savefig("explicacion_modelo.png", dpi=150)` guarda el gráfico (**antes** de `plt.show()`).

   Con los datos de práctica salen, aproximadamente, `antelacion_dias` 0,103, `deposito` 0,059, `canal` 0,042 y `tarifa` 0,037. Escribid **2–3 frases de lectura** en la justificación: «La variable más importante es la antelación, lo que tiene sentido porque, en los datos, se cancela el 30 % de las reservas con poca antelación y el 67 % de las de más de 120 días». El gráfico solo dice qué pistas usa el modelo, **no** que sean la causa. Si los datos son tabulares y queréis ir más allá, se puede usar SHAP, y con imágenes, Grad-CAM; se profundiza en el sprint 16.

4. **La revisión con la rúbrica (10 minutos).** Quien revisa **no escribió** esa parte y sigue los cinco pasos de «Cómo se revisa» de entrega y rúbrica: ejecutar `03_modelos.ipynb` de arriba abajo y comparar las cifras con la tabla de la justificación; mirar el registro (*baseline* en la fila 1, columnas rellenas, una fila de cada persona); buscar fugas y trampas; leer la justificación como el cliente y poner un nivel de 1 a 4 en cada criterio **con una frase de prueba**; y anotar el resultado. Añadid al final de `justificacion_candidato.md` una sección como esta:

```
## Revisión con la rúbrica
**Revisado por:** <nombre> · **Fecha:** <dd/mm/aaaa>

| Criterio | Nivel (1–4) | Prueba: una frase de lo que has comprobado |
|---|---|---|
| Corrección técnica | | |
| Reproducibilidad | | |
| Análisis y comunicación al cliente | | |
| IA responsable | | |

**Lo primero que mejoraríamos:** <una cosa concreta>
```

5. **Subir los ficheros (la vía del curso, desde el navegador).** Seguid Cómo entregar en GitHub. Descargad del panel de Colab (clic derecho sobre cada archivo → *Descargar*) `registro.csv`, `modelo_candidato.joblib`, `metadatos.json` y `explicacion_modelo.png`; y el cuaderno con *Archivo → Descargar → Descargar .ipynb*. Después: (1) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-15`**; (2) *Add file → Upload files*; (3) arrastrad los ficheros; (4) escribid un mensaje corto y pulsad **Commit changes**. Cada subida es un *commit* con el nombre de quien la hace. Un reparto posible: la persona A sube `03_modelos.ipynb` («Añado cuaderno de modelos»), la B sube `registro.csv` y `explicacion_modelo.png`, la C sube `justificacion_candidato.md`, `modelo_candidato.joblib` y `metadatos.json`. Los nombres de archivo, **en minúsculas y sin espacios**. Cada persona sube además su `autoevaluacion_nombre.md`.

**Qué deberíais ver.** En la carpeta `sprint-15` del repositorio, los seis ficheros del equipo y una hoja de autoevaluación por persona (**6 + N**); y en el historial (el icono del reloj), al menos tres *commits* con nombres distintos.

**Hecho cuando:** el test se ha evaluado una vez, el modelo y su explicación están guardados, la sección «Revisión con la rúbrica» está rellena y todo está en `sprint-15/`.

**Después, 60 minutos de review y retro.** Cada persona sube su autoevaluación y coevaluación a `sprint-15/` (en el tiempo de entrega). En la *review*, cada equipo enseña en 5 minutos: (1) la tabla de modelos con media ± desviación, (2) el candidato elegido y **por qué** (no solo la métrica), (3) una fuga que evitó o detectó, (4) una hipótesis sobre por qué gana ese modelo y (5) su registro de experimentos. El resto de equipos pregunta: *«¿cómo sabéis que la diferencia no es ruido?»*

### Si algo falla

| Veis... | Qué significa | Qué hacer |
|---|---|---|
| `NameError: name 'final' is not defined` | La celda del test no se ejecutó antes de guardar o explicar. | Ejecutad desde la primera celda: *Entorno de ejecución → Ejecutar todo*. |
| `TypeError: Object of type int64 is not JSON serializable` (o `float32`) | Hay un número de NumPy dentro de `metadatos` y JSON no sabe escribirlo. | Convertidlo con `float(...)` o `int(...)`, como en el código de arriba. |
| `FileNotFoundError: [Errno 2] No such file or directory: 'modelo_candidato.joblib'` | Se reinició Colab y el archivo guardado se perdió, o el nombre no coincide. | Volved a ejecutar la celda que guarda el modelo y descargad los archivos antes de cerrar. |
| El test sale mucho peor que la validación | Sobreajuste o fuga. | No cambiéis de candidato por verlo: anotad la diferencia en la justificación y explicad la causa probable. |
| `ModuleNotFoundError: No module named 'shap'` | SHAP no está instalado en la sesión. | Usad la importancia por permutación de este paso, que no necesita instalar nada. |
| El navegador no deja subir el modelo | Pesa más de 25 MB. | No lo subáis por el navegador: avisad al docente y dejad escrito en `metadatos.json` cómo volver a generarlo con vuestro cuaderno. |

## Ayuda 1 · Cómo usar la IA en este reto

La IA puede ahorrar tiempo, pero **da código que parece funcionar y no se entiende**, y puede pasar por alto una fuga o ver una que no existe. Reglas:

- **Todo lo que dé, lo ejecutáis y lo entendéis.** Si el docente pregunta por una línea, tenéis que saber explicarla.
- **El test se usa una sola vez, con el modelo ya elegido.** Elegir por el test invalida el resultado.
- **Declaradlo** en `justificacion_candidato.md` y **marcad en el registro** las pruebas hechas con código generado por IA.
- **Nada de datos personales ni claves** en una IA ni en el repositorio.

| Para qué | Ejemplo de petición | Qué comprobáis |
|---|---|---|
| Entender un modelo | «Explícame cuándo conviene un modelo de boosting frente a una regresión logística, con un ejemplo sencillo» | Que la explicación encaja con **vuestros datos** |
| Depurar un error | «Este código de scikit-learn da este error: *[error]*. Datos: *[estructura]*» | Que entendéis la causa |
| Detectar fugas | «Aquí está mi pipeline: *[código]*. ¿Hay alguna operación que use el test o el futuro?» | Que **verificáis** la respuesta; la IA puede pasarlas por alto o ver fugas inexistentes |
| Interpretar SHAP | «Tengo este gráfico SHAP (descripción). ¿Qué significa que la variable X tenga ese efecto?» | Contrastadlo con el sentido del negocio |

## Ayuda 2 · Errores frecuentes (y cómo evitarlos)

| Error | Por qué ocurre | Cómo evitarlo |
|---|---|---|
| Elegir el modelo mirando el test | «Para ver cuál es mejor» | Elegid solo con validación; el test, una vez |
| Resultados «increíbles» (99 %) | Fuga de datos | Revisad variables y partición con la lista del paso 3 |
| Registro con 3 filas de una sola persona | El responsable técnico lo hace todo | Cada persona prueba algo y lo anota con su nombre |
| Cada persona tiene su propio `registro.csv` | Se trabaja en copias distintas del cuaderno | Juntad todas las filas en un único `registro.csv` antes de subir |
| Comparar con particiones distintas | Cada uno prueba por su lado | Una partición común, guardada y compartida |
| Mejor número = mejor modelo | Se mira solo la métrica | Añadid tiempo, simplicidad y explicabilidad |
| Mezclar fotos del mismo objeto entre entrenamiento y test | En imágenes es fácil | Particionad por objeto o por parcela |
| Cambiar la semilla hasta que salga un resultado bonito | Prisa o ganas de ganar | Una semilla para todo el equipo; si cambia mucho, el modelo es inestable y hay que decirlo |

## Ayuda 3 · Si os atascáis

1. **No sabemos qué modelos probar:** repasad el *leaderboard* del Sprint 8 y la red del Sprint 10, y usad las mismas familias.
2. **Dos modelos dan casi lo mismo:** perfecto; elegid el más simple y explicadlo.
3. **No tenemos GPU o va lenta:** usad un subconjunto de los datos y anotad el coste en el registro.
4. **El modelo guardado pesa más de 25 MB:** no lo subáis por el navegador; avisad al docente, subid `metadatos.json` y dejad escrito en él cómo volver a generar el modelo con vuestro cuaderno.
5. Preguntad al equipo y, después, al docente. Más ayuda: Cómo se trabaja un reto.